Detailed Description of the Feature-Vector Elements Used in the SVM-Based Detection Approach

Each microservice pattern is characterized by a set of behavioral constraints that can be used for validation. To reduce the effect of structural noise in graphs of large-scale systems, a set of constraints is defined for every microservice pattern to describe its expected behavior. These constraints are used when initializing the pattern feature vectors in a microservice-based system.

For example, the API Gateway pattern is associated with the following behavioral constraints:

(1) The API Gateway microservice must have at least one incoming path from a Client (Client → API Gateway).

(2) The API Gateway must not be invoked by internal Worker services.

(3) The API Gateway must not directly invoke the Client.

(4) The Client must not communicate directly with internal microservices. (5) The API Gateway microservice's dependency list must contain a tool or library associated with the pattern, such as Zuul.

The 30-element feature vector defines three principal roles for the microservices in the system graph. Each vector therefore represents the characteristics of three microservices: a Master, a Worker, and a Client. Ten vector elements are allocated to each role.

The ten elements assigned to a role are designed to encode the behavioral constraints associated with a pattern. For example, among the first ten elements, which correspond to the Master, the first two—InEdge and OutEdge—score the Master's supported and invoked operations. The remaining eight elements analyze and encode the Master's relationships with the other two roles, Worker and Client.

To encode communication links, a prime number is assigned to each HTTP method: GET:2, POST:3, PUT:5, and DELETE:7. When the implementation of a microservice requires a particular tool or library in its dependency file (for example, pom.xml), an additional prime number is assigned to that dependency. Thus, because an API Gateway commonly uses Zuul in real systems, the prime 11 may be used to represent this dependency.

The following example explains the feature-vector initialization process, the interpretation of each element, and the differences among feature-vector values when the system graph contains several structurally similar subgraphs. It demonstrates how behavioral rules can reduce structural noise effectively and thereby substantially decrease the false-positive rate in pattern detection.

Example. Assume that Figure 1 presents part of the graph of a microservice-based system. The graph can be decomposed into three structurally similar subgraphs, as shown in Figures 2–4. Each subgraph must therefore be evaluated to determine whether it satisfies the behavioral conditions and constraints of the API Gateway pattern.

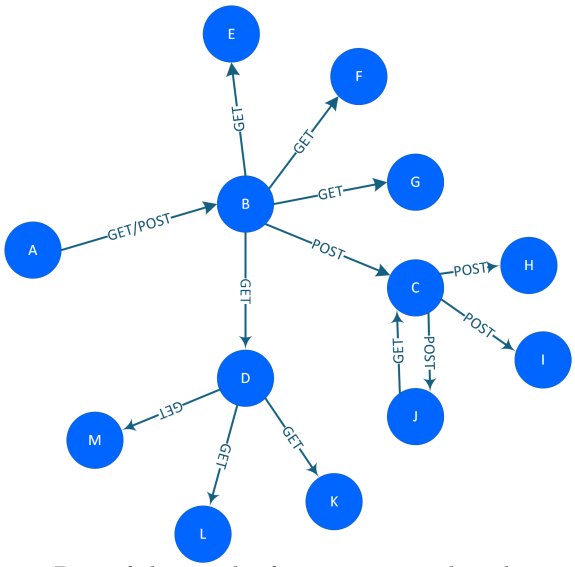

Fig. 1: Part of the graph of a microservice-based system.

In Subgraph 1 (Figure 2), microservice B acts as the Master: it receives one or more requests from Client microservice A and forwards them to microservices E, F, G, C, and D, which act as Workers. The API Gateway constraints are evaluated as follows:

(1) Microservice B receives at least one request from microservice A.

(2) None of the Worker microservices sends a request to microservice B.

(3) Microservice B does not invoke microservice A.

(4) No direct communication exists between microservice A and the Worker microservices.

(5) The dependency list of microservice B contains the Zuul library.

Subgraph 1 is therefore an instance of the API Gateway pattern.

In Subgraph 2 (Figure 3), microservice D receives a request from microservice B and forwards it to microservices K, L, and M. The API Gateway constraints are evaluated as follows:

(1) Microservice D, acting as the Master, receives at least one request from microservice B, acting as the Client.

(2) None of microservices K, L, and M, acting as Workers, invokes microservice D.

(3) Microservice D does not invoke microservice B.

(4) No direct communication exists between microservice B and microservices K, L, and M.

(5) Inspection of microservice D's dependency file shows that it contains no tool or library associated with the API Gateway pattern.

Consequently, Subgraph 2 is not an API Gateway instance. It satisfies four of the five API Gateway constraints and violates only the fifth. Although it is structurally very similar to the API Gateway pattern, it represents the

Aggregator pattern instead. Including the fifth constraint distinguishes Subgraphs 1 and 2 despite their structural similarity and prevents a false positive when Subgraph 2 is analyzed.

In Subgraph 3, microservice C is treated as the Master, microservice B as the Client, and microservices H, I, and J as Workers. The API Gateway constraints are evaluated as follows:

(1) Microservice C receives a request from microservice B.
(2) Microservice C is invoked by microservice J, which acts as a Worker; this violates an API Gateway constraint.
(3) Microservice C does not invoke microservice B.
(4) No direct communication exists between microservice B and microservices H, I, and J.
(5) Microservice C's dependency list contains no tool or library associated with the API Gateway pattern.

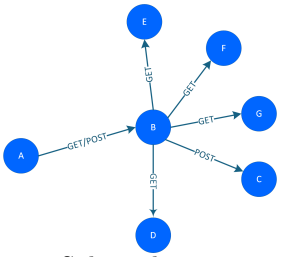

Fig. 2: Subgraph 1 extracted from the system graph.

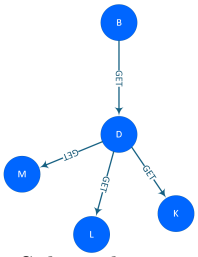

Fig. 3: Subgraph 2 extracted from the system graph.

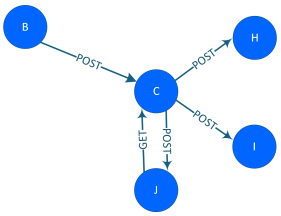

Fig. 4: Subgraph 3 extracted from the system graph.

Evaluation of all five constraints shows that Subgraph 3 violates Constraints 2 and 5 and therefore does not match the API Gateway pattern. Table 1 presents the value of every feature-vector element for the three subgraphs and illustrates the role assignments.

- Subgraph 1: nodes A, B, and E act as Client, Master, and Worker, respectively.
- Subgraph 2: nodes B, D, and K act as Client, Master, and Worker, respectively.
- Subgraph 3: nodes B, C, and J act as Client, Master, and Worker, respectively.

The feature vectors for Subgraphs 1, 2, and 3 are defined below. They represent the Master, Worker, and Client characteristics in each subgraph and are used for pattern validation:

$$\text{Subgraph } 1 = [66, 66, 1, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 6, 1, 1, 0, 0, 0, 0, 0, 0],$$
$$\text{Subgraph } 2 = [2, 2, 1, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 1, 0, 0, 0, 0, 0, 0, 0],$$
$$\text{Subgraph } 3 = [3, 3, 0, 1, 0, 0, 0, 0, 0, 0, 3, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0, 3, 0, 1, 0, 0, 0, 0, 0, 0].$$

Table 1: Description and initialization of the feature-vector elements.

Microservice role	Feature name	Description	*SG_1*	*SG_2*	*SG_3*
Master	M_InEdge	Identifies the HTTP methods exposed by the Master microservice's controller. The final value is the product of the weights assigned to the exposed methods and required pattern-specific dependencies.	$2 \times 3 \times 11 = 66$ (GET:2, POST:3, Zuul:11)	2	3
	M_OutEdge	Identifies the HTTP methods through which the Master invokes other microservices. The final value is the product of the weights associated with those methods and dependencies.	$2 \times 3 \times 11 = 66$	2	3
	M_GET_W	Equals 1 if the Master is permitted to invoke a Worker using GET; otherwise, it equals 0.	1	1	0
	M_POST_W	Equals 1 if the Master is permitted to invoke a Worker using POST; otherwise, it equals 0.	0	0	1
	M_PUT_W	Equals 1 if the Master is permitted to invoke a Worker using PUT; otherwise, it equals 0.	0	0	0
	M_DELETE_W	Equals 1 if the Master is permitted to invoke a Worker using DELETE; otherwise, it equals 0.	0	0	0
	M_GET_C	Equals 1 if the Master is permitted to invoke the Client using GET; otherwise, it equals 0.	0	0	0
	M_POST_C	Equals 1 if the Master is permitted to invoke the Client using POST; otherwise, it equals 0.	0	0	0
	M_PUT_C	Equals 1 if the Master is permitted to invoke the Client using PUT; otherwise, it equals 0.	0	0	0
	M_DELETE_C	Equals 1 if the Master is permitted to invoke the Client using DELETE; otherwise, it equals 0.	0	0	0
Worker	W_InEdge	Identifies the HTTP methods exposed by the Worker microservice's controller. The final value is the product of the method weights.	2	2	3
	W_OutEdge	Identifies the HTTP methods through which the Worker invokes other microservices.	0	0	2
	W_GET_M	Equals 1 if the Worker is permitted to invoke the Master using GET; otherwise, it equals 0.	0	0	1
	W_POST_M	Equals 1 if the Worker is permitted to invoke the Master using POST; otherwise, it equals 0.	0	0	0
	W_PUT_M	Equals 1 if the Worker is permitted to invoke the Master using PUT; otherwise, it equals 0.	0	0	0
	W_DELETE_M	Equals 1 if the Worker is permitted to invoke the Master using DELETE; otherwise, it equals 0.	0	0	0

Continued on the next page.

Table continued from the previous page.

Microservice role	Feature name	Description	SG_1	SG_2	SG_3
	W_GET_C	Equals 1 if the Worker is permitted to invoke the Client using GET; otherwise, it equals 0.	0	0	0
	W_POST_C	Equals 1 if the Worker is permitted to invoke the Client using POST; otherwise, it equals 0.	0	0	0
	W_PUT_C	Equals 1 if the Worker is permitted to invoke the Client using PUT; otherwise, it equals 0.	0	0	0
	W_DELETE_C	Equals 1 if the Worker is permitted to invoke the Client using DELETE; otherwise, it equals 0.	0	0	0
Client	C_InEdge	Identifies the HTTP methods exposed by the Client microservice's controller.	0	0	0
	C_OutEdge	Identifies the HTTP methods through which the Client invokes other microservices.	6	2	3
	C_GET_M	Equals 1 if the Client is permitted to invoke the Master using GET; otherwise, it equals 0.	1	1	0
	C_POST_M	Equals 1 if the Client is permitted to invoke the Master using POST; otherwise, it equals 0.	1	0	1
	C_PUT_M	Equals 1 if the Client is permitted to invoke the Master using PUT; otherwise, it equals 0.	0	0	0
	C_DELETE_M	Equals 1 if the Client is permitted to invoke the Master using DELETE; otherwise, it equals 0.	0	0	0
	C_GET_W	Equals 1 if the Client is permitted to invoke a Worker using GET; otherwise, it equals 0.	0	0	0
	C_POST_W	Equals 1 if the Client is permitted to invoke a Worker using POST; otherwise, it equals 0.	0	0	0
	C_PUT_W	Equals 1 if the Client is permitted to invoke a Worker using PUT; otherwise, it equals 0.	0	0	0
	C_DELETE_W	Equals 1 if the Client is permitted to invoke a Worker using DELETE; otherwise, it equals 0.	0	0	0